TALLER PRÁCTICO DE REPASO Y CONSOLIDACIÓN EN C++

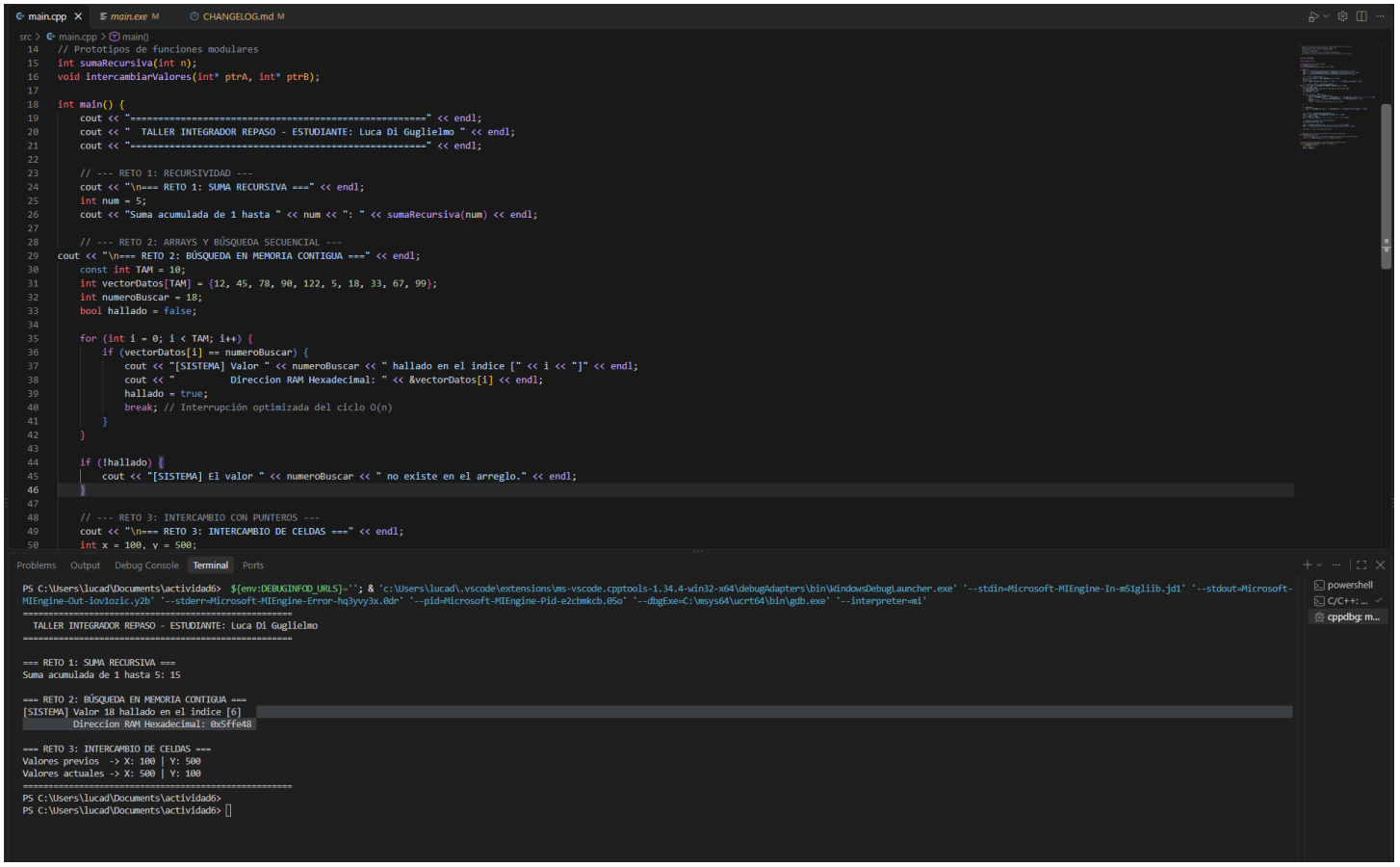
Luca Di Guglielmo Villa
E.E.S.T N°1 de Vicente López
Curso: Laboratorio de Programación 5°3°
Profesor: Ganduglia Yamil & York Elias Mansilla Muñoz
Fecha: 14/10/2026

Introducción

En la actividad elaborada, se busca repasar y consolidar muchos conceptos clave de la programación en C + + y de bajo nivel, entre ellos están: la Recursividad, la Búsqueda Secuencial, punteros y su intercambio físico

Desarrollo

Una vez creado las bases del repositorio, pegue el código esqueleto en el archivo main.cpp, lo compile y lo ejecute. Esto resultó en la salida de los 3 ‘retos’ de la actividad.



En el **primer reto**, el programa realiza una suma recursiva sobre 1 que se acumula hasta un valor establecido por el programa (5). En el **segundo reto**, se muestra en pantalla un número el cual el programa busca dentro de un rango (array) de otros enteros. En el **tercer reto**, se realiza un intercambio físico de dos valores, mostrando el estado previo y posterior a la acción.

Actividades:

1. Si una función recursiva carece de caso base, se produce un *stack overflow* (desbordamiento de pila). La función se llama a sí misma indefinidamente, consumiendo espacio en la pila de llamadas hasta agotar la memoria reservada para ella, lo que provoca que el programa se cierre abruptamente con un error de segmentación o violación de acceso.

2. La instrucción break optimiza el rendimiento porque, al encontrar el elemento buscado, termina el ciclo inmediatamente. Sin ella, el bucle continuaría iterando sobre los elementos restantes del arreglo de forma innecesaria, desperdiciando ciclos de CPU. En un arreglo de N elementos, el peor caso pasa de O(N) a O(k) donde k es la posición del elemento encontrado.

3. Se puede comprobar con &array[i] y &array[i+1]. Si la diferencia entre ambas direcciones es

exactamente sizeof(tipo_del_elemento), los elementos son contiguos. Por ejemplo, en un int array[5], &array[1] - &array[0] debe ser igual a 4 bytes (tamaño de int), confirmando que ocupan celdas consecutivas en RAM.

4. La variable temporal (temp) preserva el valor original de una variable durante el intercambio. En un swap con punteros: *temp = *a; *a = *b; *b = *temp;. Sin ella, al asignar *a = *b se perdería el valor original de *a, haciendo imposible completar el intercambio correctamente.

5. El reto que más difícil se me hizo entender fue el primero, ya que todavia no termine de entender la lógica detrás de él

Conclusión

A lo largo del desarrollo, se aplicaron y desarrollaron los conceptos clave de la actividad de una forma práctica no convencional, en vez de crear el codigo, el codigo ya fue proporcionado de forma previa y yo debía hacer un manual de programador y usuario, lo que me forzaba a entender estos conceptos de programación de bajo nivel

Referencias:

Luca Di Guglielmo

Github: https://github.com/specIux/actividad-6-LPR

Gitlab: https://gitlab.com/specIux/actividad-6-lpr

Laboratorio de Programación 5°3°

Actividad: 📺🛠️🖌️ ACTIVIDAD 6: TALLER PRÁCTICO DE REPASO Y CONSOLIDACIÓ…